Ausgabe auf der Konsole

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Unterstützung beim Programmieren durch IntelliJ
- Debugging
- KI-Unterstützung

Was wird in diesem Foliensatz besprochen?

- Ausgabe auf der Konsole
- String-Klasse
- Formatierte Ausgabe

Was lernen Sie kennen?

- Ausgabe auf der Konsole
- Aufbau von Strings
- String-Methoden
- Spezielle Formatierungsmöglichkeiten

Ausgabe (Wiederholung)

Ausgabe auf der Kommandozeile (Konsole)

- System.out.print()
- System.out.println()

Kann für unterschiedliche Datentypen verwendet werden

String-Klasse (Wiederholung)

Klasse String ist in Java vordefiniert

Ähnlichkeiten zu einfachen Datentypen

- String – Literale wie z. B. "TestString"
- Es gibt einen Operator: + (und auch +=)
- Implizite Typkonvertierung

Aber: String ist eine Klasse

Klassen und Objekte (ein einfacher Ausblick)

Klasse
• Beschreibt die gemeinsamen Eigenschaften und Operationen einer Menge gleichartiger Objekte • Stellt einen Konstruktionsplan für Objekte dar

Objekt
• Ist eine konkrete Instanz einer Klasse • Es kann mehrere Objekte von einer Klasse geben • Struktur und Verhalten der Objekte einer Klasse sind gleich

Klasse / Objekt(e)

```java
public final class String … {

    private final byte[] value;

    private final byte coder;
    …

    public String() {
        this.value = "".value;
        this.coder = "".coder;
    }

    …

    public String(char value[]) {
        this(value, 0, value.length, null);
    }
    …

    public boolean isEmpty() {
        return value.length == 0;
    }
    …
}
```

Klasse gibt Aufbau und Operationen vor

"Hello"

"World"

"EP1"

Jedes Objekt hat „seine“ **eigenen** Daten (Text), aber es können auf alle Objekte **dieselben Operationen** ausgeführt werden

Strings (String-Objekte) erzeugen

```
String s1 = "Hello";
```

Bisher verwendet

```
String s2 = new String("Hello");
```

Anderes Objekt erzeugen

```
String s3 = new String(s1);
```

Anderes Objekt erzeugen

Konstruktor der Klasse String aufrufen

Datentypen in Java

Einfache (primitive) Datentypen

- byte, short, int, long, float, double, char, boolean

Referenztypen

- Klassen
- Interfaces
- Arrays

Datentypen (Wiederholung)

Variablen haben in Java einen Datentyp

Gibt an, wie die Daten im Speicher interpretiert werden sollen

- Beispiel: int-Variable mit 32-Bit -> 32 Bit als ganze Zahl auffassen

Und bei einer Variable eines Referenztyps?

- Im Speicher der Variable existiert nur eine Referenz auf die Daten
- Daten liegen an einer anderen Stelle im Speicher

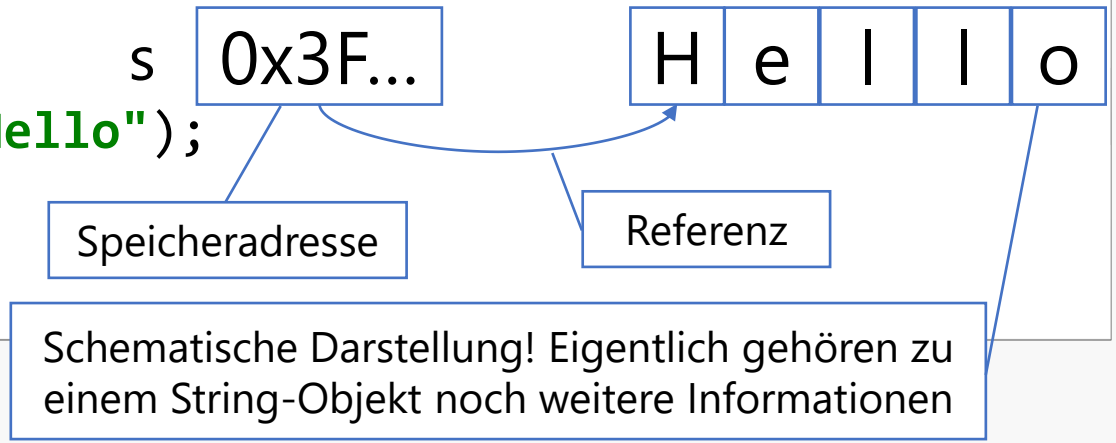
Beispiel mit int und String

int-Variable (schematisch)

```
int x = 1;
```

String Variable (schematisch)

```
String s = new String("Hello");
```



Besonderheit bei String

String-Objekte sind nach dem Anlegen **nicht** mehr veränderbar

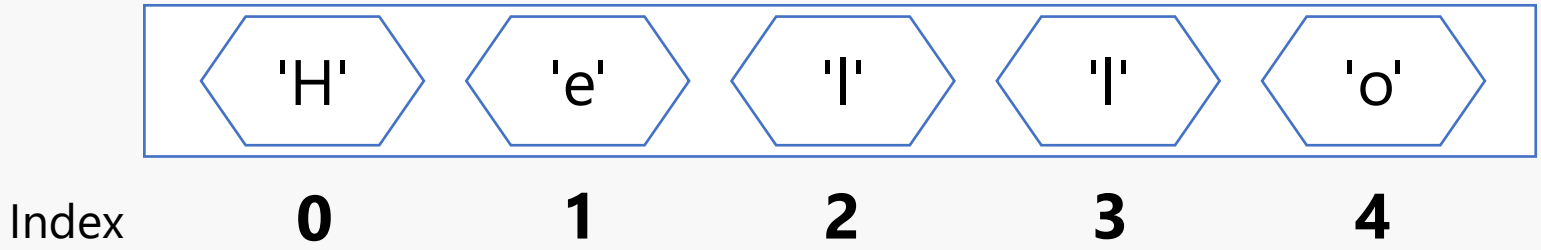
- Bei jeder Konkatenation mit + wird ein neuer String (neues String-Objekt) erzeugt

String-Literal kommt nur einmal im Programm vor

```
String s1 = "Hello";
String s2 = "Hello";
```

Beide Strings verweisen auf denselben Speicherbereich mit dem Inhalt „Hello“

Struktur eines Strings (Beispiel)



Index
• Jedes Zeichen besitzt einen Index (Position im String) • Von **0** bis **(Länge des Strings) - 1** • Ermöglicht Zugriff auf einzelne Zeichen

Operationen auf Strings

+ ist eine Ausnahme

- Keine weiteren Operatoren möglich

Weitere Operationen?

- Methoden, die auf ein Objekt aufgerufen werden können

Java-API

- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html
- Aufrufschema
  - Stringvariable gefolgt von einem „.“ und danach kommt ein Methodenname (eventuell Parameter)
- Beispiel

```
String s = "This is a test string!";
System.out.println(s.charAt(1));
```

Einige hilfreiche String-Methoden

```java
public class StringTest {

    public static void main(final String[] args) {
        String s = "This is a test string!";
        System.out.println(s.length());
        System.out.println(s.isEmpty());
        System.out.println(s.charAt(2));
        System.out.println(s.startsWith("T"));
        System.out.println(s.indexOf('s'));
        System.out.println(s.lastIndexOf('s'));
        System.out.println(s.indexOf('s', 5));
        System.out.println(s.lastIndexOf('s', 14));
        System.out.println(s.indexOf("est"));
        System.out.println(s.replace('i', 'e'));
        System.out.println(s.substring(8));
        System.out.println(s.substring(5, 15));
        System.out.println(s.toLowerCase());
    }

}
```

```
22
false
i
true
3
15
6
12
11
Thes es a test streng!
a test string!
is a test
this is a test string!
```

Schleifen und Strings

```java
String s = "Hello";
for (int i = 0; i < s.length(); i++) {
    System.out.println(s.charAt(i));
}
System.out.println("#");
```

```
H
e
l
l
o
#
```

```java
String s = "Hello";
String result = "";
for (int i = 0; i < s.length(); i++) {
    result += " # " + s.charAt(i);
}
result += " # ";
System.out.println(result);
```

```
# H # e # l # l # o #
```

Vorsicht bei Indexzugriff

```java
String s = "Hello";
for (int i = 0; i <= s.length(); i++) {
    System.out.println(s.charAt(i));
}
System.out.println("#");
```

```
H
e
l
l
o
Exception in thread "main" java.lang.StringIndexOutOfBoundsException: String index out of range: 5
       at java.base/java.lang.StringLatin1.charAt(StringLatin1.java:48)
       at java.base/java.lang.String.charAt(String.java:1515)
       at Test.main(Test.java:12)
```

Ungültiger Indexwert -> Ausnahme (Exception) wird erzeugt (geworfen)

Formatierte Ausgabe – einfaches Beispiel

Mit den Methoden format (Rückgabe) und printf (Ausgabe) kann ein formatierter String erzeugt werden

Achtung: Das ist eine statische Methode (benötigt kein Objekt)

```
int x = 123456;
double y = 2947365.28;
char c = '#';
String text = String.format("Inhalt: x = %10d  y = %15.5f  c = %5c", x, y, c);
System.out.println(text);
System.out.printf("Inhalt: x = %0+10d  y = %10.1f  c = %c", x, y, c);
```

```
Inhalt: x =     123456  y =   2947365,28000  c =     #
Inhalt: x = +000123456  y =  2947365,3  c = #
```

Formatierte Ausgabe – Format spezifizieren

Beispiel von vorher – was wird wie formatiert

```
String.format("Inhalt: x = %10d  y = %15.5f  c = %5c", x, y, c);
```

Sehr viele Möglichkeiten für den Aufbau

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Formatter.html

String-Textblöcke (seit Java 15)

Kompaktere Darstellung von Strings, die über mehrere Zeilen gehen

```
String s = "line 1\nline 2\nline 3\n";

String t = """
line 1
line 2
line 3
""";
```

Aufbau und Einschränkungen

https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html#jls-3.10.6